BÁO CÁO THỰC HÀNH LAB 02

Viết Device Driver & Quản lý File System (Kernel Module · Character Device · sysfs/procfs · MTD/JFFS2)

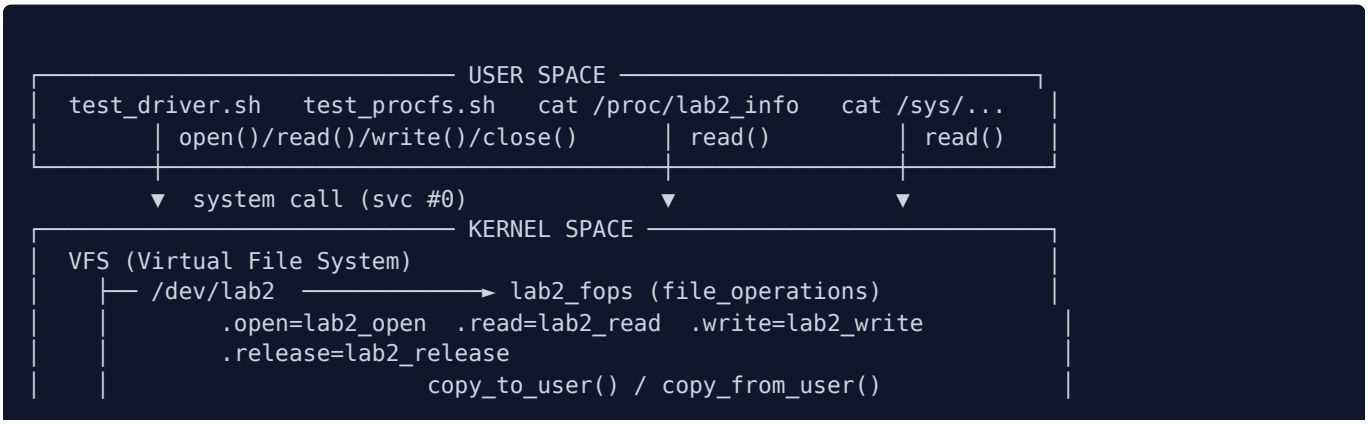
Mục 1: Thông tin sinh viên & Môi trường thực hiện

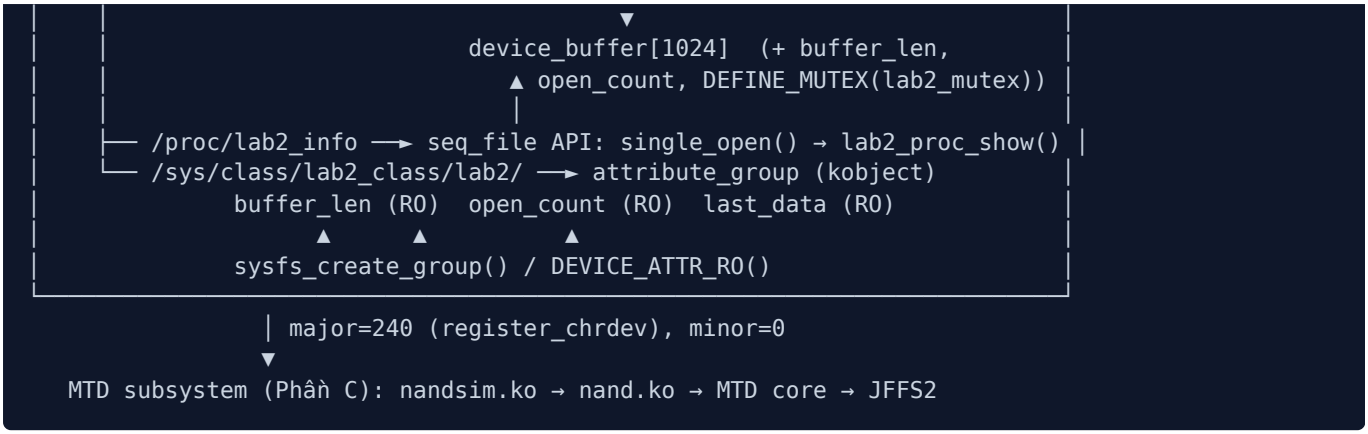
Họ và tên	Vũ Tùng Minh
Mã số sinh viên (MSSV)	SE201043
Lớp chuyên ngành	AS2105 (Môn: LOS201 – Embedded Linux Systems)
Thời gian thực hiện	17/10/2026
Kiến trúc mục tiêu	ARMv7-A – Cortex-A9 (QEMU `vexpress-a9`, 2 core, 512 MB RAM)
Kernel target	Linux **5.15.0** (`#2 SMP ... armv7l`), cấu hình mở rộng thêm `CONFIG_MTD_RAW_NAND=m`, `CONFIG_MTD_NAND_NANDSIM=m` cho Phần C
Toolchain	`arm-linux-gnueabihf-gcc (Ubuntu 11.4.0-1ubuntu1~22.04.3) 11.4.0` – hard-float ABI
Môi trường host	CachyOS Linux x86_64 (kernel 7.2.9) + container **Ubuntu 22.04 LTS** (Podman); QEMU **6.2.0**
Công cụ	mtd-utils **2.1.4**, BusyBox **1.35.0** (static, có `flash_eraseall`, `nandwrite`, `nanddump`), `fakeroot`, `cpio`

> **Ghi chú môi trường Phần C:** Host CachyOS đang chạy kernel 7.2.9-1 nhưng **không có thư mục module tương ứng** (`/lib/modules/7.2.9-1*` trống) và tài khoản không có quyền `root`, nên **không thể `modprobe nandsim` trên host** như hướng dẫn. Để vẫn thực hiện đúng bản chất kỹ thuật, em đã **biên dịch `nandsim.ko` kèm kernel target** và chạy toàn bộ quy trình MTD → JFFS2 **ngay trong QEMU ARM**. Trên `vexpress-a9`, NOR flash và PSRAM của bo mạch đã chiếm `mtd0`/`mtd1`, nên NAND simulator là **`mtd2`** (đúng như lưu ý trong đề bài: dùng `/proc/mtd` để xác định đúng phân vùng). Tất cả số liệu trong báo cáo đều là kết quả chạy thật, không mô phỏng lại.

Mục 2: Sơ đồ kiến trúc Driver

Luồng dữ liệu từ không gian người dùng xuống driver và ra các giao diện quản trị:





Luồng ghi dữ liệu (write)

```
user buffer (địa chỉ ảo user) → copy_from_user(device_buffer, buf, len)
     → [mutex_lock] ghi vào device_buffer → buffer_len = len
     → device_buffer[len] = '\0' → [mutex_unlock]
     → procfs/sysfs đọc lại cùng device_buffer khi người dùng cat
```

Cây sysfs của driver

```
/sys/class/lab2_class/lab2/
├── buffer_len     (0444, DEVICE_ATTR_RO)  -> sô´byte dữ liệu hiện có
├── open_count     (0444, DEVICE_ATTR_RO)  -> sô´lần open() tích lũy
├── last_data      (0444, DEVICE_ATTR_RO)  -> nội dung device_buffer
├── dev            -> ánh xạ major:minor (240:0)
├── power/ , subsystem/ , uevent            (do kernel device model tạo)
```

Mục 3: Log thực hành và phân tích ảnh chụp

Phần A – Kernel Module Character Device

Module được build trực tiếp bởi hệ thống Kbuild của kernel ARM đã build ở LAB-01 (không dùng gcc thường), với `CC="arm-linux-gnueabihf-gcc -mfloat-abi=soft"` đúng như kernel ARM yêu cầu. `modinfo` xác nhận `vermagic: 5.15.0 SMP mod_unload ARMv7 p2v8` – khớp chính xác kernel target, đây là điều kiện bắt buộc để `insmod` thành công.

```
PHAN A – Build module ARM + modinfo (host Ubuntu 22.04)
$ cd ~/embedded_lab2/driver
$ make clean
  CLEAN   /root/embedded_lab2/driver/Module.symvers
make[1]: Leaving directory '/work/LAB/embedded_lab1/kernel/linux-5.15'

$ make
make -C /root/embedded_lab2/driver/../../embedded_lab1/kernel/linux-5.15 M=/root/embedded_lab2/driver ARCH=arm
 CROSS_COMPILE=arm-linux-gnueabihf- CC="arm-linux-gnueabihf-gcc -mfloat-abi=soft" modules
make[1]: Entering directory '/work/LAB/embedded_lab1/kernel/linux-5.15'
  CC [M]  /root/embedded_lab2/driver/lab2_driver.o
  MODPOST /root/embedded_lab2/driver/Module.symvers
  CC [M]  /root/embedded_lab2/driver/lab2_driver.mod.o
  LD [M]  /root/embedded_lab2/driver/lab2_driver.ko
make[1]: Leaving directory '/work/LAB/embedded_lab1/kernel/linux-5.15'

$ ls -lh lab2_driver.ko
-rw-r--r-- 1 root root 141K Oct 17 00:17 lab2_driver.ko

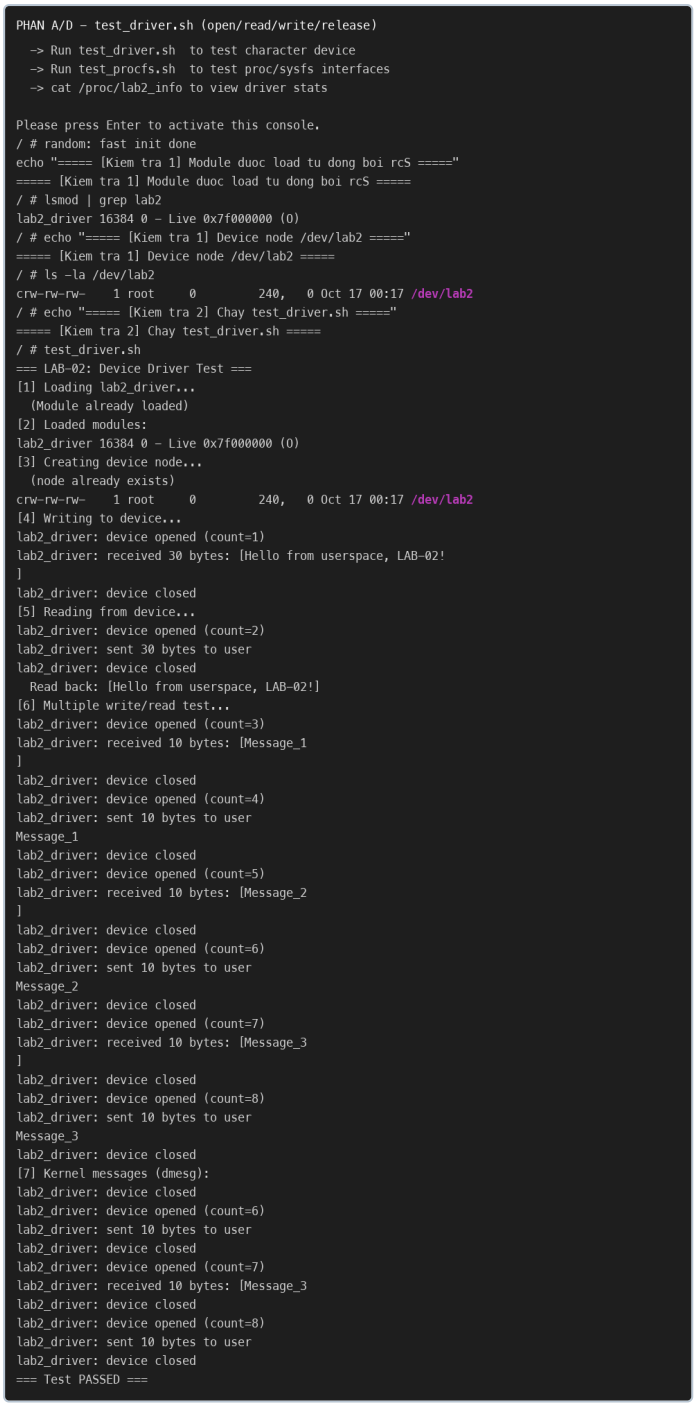
$ file lab2_driver.ko
lab2_driver.ko: ELF 32-bit LSB relocatable, ARM, EABI5 version 1 (SYSV), BuildID[sha1]=e2937b1007cbe70be0e1b77
78588831563780b82, with debug_info, not stripped

$ modinfo lab2_driver.ko
filename:       /root/embedded_lab2/driver/lab2_driver.ko
version:        1.0
description:    LAB-02: Character Device Driver with procfs and sysfs interfaces
author:         Vu Tung Minh – SE201043
license:        GPL
srcversion:     AE8325818819D5115761363
depends:
name:           lab2_driver
vermagic:       5.15.0 SMP mod_unload ARMv7 p2v8

$ modinfo lab2_driver.ko | grep vermagic
vermagic:       5.15.0 SMP mod_unload ARMv7 p2v8
```

Hình 1: `make` sinh `lab2_driver.ko` (ELF 32-bit ARM EABI5), `file` và `modinfo` xác nhận license GPL, author và vermagic 5.15.0.

Trong QEMU, module được `rcS` nạp tự động. Script `test_driver.sh` kiểm tra lần lượt open → write → read → release; kernel log (`dmesg`) in đúng vòng đời module và số byte nhận/gửi, chứng minh `copy_from_user()`/`copy_to_user()` hoạt động và `open_count` tăng tuần tự (1→8) không bị race trên bộ đệm.



Hình 2: Kết quả `test_driver.sh` – ghi 30 byte, đọc lại đúng `[Hello from userspace, LAB-02!]`, vòng lặp write/read 3 lần in lại Message_1..3, kết thúc `=== Test PASSED ===`.

Phần B – Giao tiếp qua procfs (/proc) và sysfs (/sys)

Entry `/proc/lab2_info` dùng seq_file API (`single_open()`) xuất thống kê dạng text; ba thuộc tính sysfs (`buffer_len`, `open_count`, `last_data`) dùng macro `DEVICE_ATTR_RO`, mỗi file chỉ chứa một giá trị. Trước khi ghi, `Data length = 0` và `Last data = (empty)`; sau khi ghi 30 byte, procfs và sysfs đều phản ánh đúng dữ liệu mới.

```
PHAN B – procfs + sysfs
  -> Run test_procfs.sh  to test proc/sysfs interfaces
  -> cat /proc/lab2_info to view driver stats

Please press Enter to activate this console.
/ # random: fast init done
echo "===== [Kiem tra 3] procfs / sysfs ====="
===== [Kiem tra 3] procfs / sysfs =====
/ # test_procfs.sh
=== Test procfs & sysfs ===
--- /proc/lab2_info (before write) ---
LAB-02 Driver Statistics
======================
Driver name   : lab2
Major number  : 240
Buffer size   : 1024 bytes
Data length   : 0 bytes
Open count    : 0
Last data     : (empty)
lab2_driver: device opened (count=1)
lab2_driver: received 30 bytes: [Embedded Linux Lab2 Test Data
]
lab2_driver: device closed
--- /proc/lab2_info (after write) ---
LAB-02 Driver Statistics
======================
Driver name   : lab2
Major number  : 240
Buffer size   : 1024 bytes
Data length   : 30 bytes
Open count    : 1
Last data     : [Embedded Linux Lab2 Test Data
]
--- sysfs attributes ---
buffer_len : 30
open_count : 1
last_data  : Embedded Linux Lab2 Test Data
=== procfs/sysfs Test PASSED ===
```

Hình 3: `test_procfs.sh` – `/proc/lab2_info` trước/sau write, các attribute sysfs đọc được từ userspace, `=== procfs/sysfs Test PASSED ===`.

Phần C – MTD Subsystem, NAND và JFFS2

NAND 32 MiB được mô phỏng bằng `nandsim first_id_byte=0x20 second_id_byte=0x35` (SLC, page 512 B, eraseblock 16 KiB, OOB 16 B). Kernel nhận diện đúng chip *ST Micro NAND 32MiB* và tạo phân vùng `"NAND simulator partition 0"`. Vì bo mạch đã có NOR flash (`mtd0`) và PSRAM (`mtd1`), phân vùng NAND là `mtd2`.

```
PHAN C – nandsim: MTD NAND 32MiB + JFFS2 image
/ # echo "--- [C.2] Truoc khi load nandsim: cac MTD san co cua bo mach ---"
--- [C.2] Truoc khi load nandsim: cac MTD san co cua bo mach ---
/ # cat /proc/mtd
dev:    size   erasesize  name
mtd0: 08000000 00040000 "40000000.flash"
mtd1: 02000000 00001000 "48000000.psram"
/ # insmod /lib/modules/5.15.0/nand.ko
/ # insmod /lib/modules/5.15.0/nandsim.ko first_id_byte=0x20 second_id_byte=0x35
[nandsim] warning: read_byte: unexpected data output cycle, state is STATE_READY return 0x0
[nandsim] warning: read_byte: unexpected data output cycle, state is STATE_READY return 0x0
[nandsim] warning: read_byte: unexpected data output cycle, state is STATE_READY return 0x0
[nandsim] warning: read_byte: unexpected data output cycle, state is STATE_READY return 0x0
[nandsim] warning: read_byte: unexpected data output cycle, state is STATE_READY return 0x0
[nandsim] warning: read_byte: unexpected data output cycle, state is STATE_READY return 0x0
nand: device found, Manufacturer ID: 0x20, Chip ID: 0x35
nand: ST Micro NAND 32MiB 1,8V 8-bit
nand: 32 MiB, SLC, erase size: 16 KiB, page size: 512, OOB size: 16
flash size: 32 MiB
page size: 512 bytes
OOB area size: 16 bytes
sector size: 16 KiB
pages number: 65536
pages per sector: 32
bus width: 8
bits in sector size: 14
bits in page size: 9
bits in OOB size: 4
flash size with OOB: 33792 KiB
page address bytes: 3
sector address bytes: 2
options: 0x42
Scanning device for bad blocks
Creating 1 MTD partitions on "NAND 32MiB 1,8V 8-bit":
0x000000000000-0x000002000000 : "NAND simulator partition 0"
mtdblock: MTD device 'NAND simulator partition 0' is NAND, please consider using UBI block devic
es instead.
/ # echo "--- Sau khi load nandsim: 32MiB SLC NAND, page 512B, erase 16KiB ---"
--- Sau khi load nandsim: 32MiB SLC NAND, page 512B, erase 16KiB ---
/ # cat /proc/mtd
dev:    size   erasesize  name
mtd0: 08000000 00040000 "40000000.flash"
mtd1: 02000000 00001000 "48000000.psram"
mtd2: 02000000 00004000 "NAND simulator partition 0"
/ # echo "--- Xac dinh dung phan vung NAND simulator (mtd2 tren vexpress-a9) ---
"
--- Xac dinh dung phan vung NAND simulator (mtd2 tren vexpress-a9) ---
/ # grep -i nand /proc/mtd
mtd2: 02000000 00004000 "NAND simulator partition 0"
/ # ls -la /dev/mtd2 /dev/mtdblock2
crw-------    1 root     0          90,   4 Oct 17 00:11 /dev/mtd2
brw-------    1 root     0          31,   2 Oct 17 00:11 /dev/mtdblock2
/ # echo "--- [C.3] Erase toan bo NAND (dien 0xFF) ---"
--- [C.3] Erase toan bo NAND (dien 0xFF) ---
/ # flash_eraseall /dev/mtd2 > /tmp/erase.log 2>&1; tail -c 220 /tmp/erase.log;
echo ""
Erasing 16 Kibyte @ 2000000 - 100% complete.

/ # echo "--- JFFS2 image built on host: mkfs.jffs2 --eraseblock=0x4000 --no-cle
anmarkers ---"
--- JFFS2 image built on host: mkfs.jffs2 --eraseblock=0x4000 --no-cleanmarkers ---
/ # ls -lh /root/rootfs.jffs2
-rw-r--r--    1 root     0          16.0K Oct 17 00:03 /root/rootfs.jffs2
/ # echo "--- Ghi JFFS2 image vao NAND bang nandwrite -p ---"
--- Ghi JFFS2 image vao NAND bang nandwrite -p ---
/ # nandwrite -p /dev/mtd2 /root/rootfs.jffs2
```

Hình 4: Load `nand.ko` + `nandsim.ko`; `/proc/mtd` xác định NAND simulator là `mtd2` (size 0x02000000 = 32 MiB, erasesize 0x4000 = 16 KiB); `flash_eraseall /dev/mtd2` xóa sạch flash rồi `nandwrite -p` ghi image JFFS2.

Sau khi `mount -t jffs2 /dev/mtdblock2 /mnt/jffs2`, các file `hello.txt`, `info.txt`, `logs/boot.log` đọc được đúng nội dung. Ghi thêm dữ liệu runtime, `umount` rồi mount lại thì **dữ liệu vẫn còn nguyên** (persistence) – đây là đặc trưng cốt lõi của JFFS2.

```
PHAN C – Mount JFFS2, ghi du lieu, persistence sau umount/mount
/ # echo "--- [C.4] Mount JFFS2 ---"
--- [C.4] Mount JFFS2 ---
/ # mkdir -p /mnt/jffs2
/ # mount -t jffs2 /dev/mtdblock2 /mnt/jffs2 && echo "MOUNT OK"
MOUNT OK
/ # mount | grep jffs2
/dev/mtdblock2 on /mnt/jffs2 type jffs2 (rw,relatime)
/ # ls -la /mnt/jffs2/
total 1
drwxr-xr-x    4 root     0                0 Jan  1  1970 .
drwxr-xr-x    3 root     0               60 Oct 17 00:13 ..
-rw-r--r--    1 root     0               18 Oct 17 00:03 hello.txt
-rw-r--r--    1 root     0               20 Oct 17 00:03 info.txt
drwxr-xr-x    2 root     0                0 Oct 17 00:03 logs
/ # echo "--- Noi dung file goc ---"
--- Noi dung file goc ---
/ # cat /mnt/jffs2/hello.txt
Hello from JFFS2!
/ # cat /mnt/jffs2/info.txt
Embedded Linux Lab2
/ # cat /mnt/jffs2/logs/boot.log
Sat Oct 17 00:03:43 UTC 2026
/ # echo "--- Ghi them du lieu runtime vao JFFS2 ---"
--- Ghi them du lieu runtime vao JFFS2 ---
/ # echo "Written at $(date)" >> /mnt/jffs2/logs/boot.log
/ # echo "New runtime file" > /mnt/jffs2/runtime.txt
/ # cat /mnt/jffs2/logs/boot.log
Sat Oct 17 00:03:43 UTC 2026
Written at Sat Oct 17 00:14:20 UTC 2026
/ # echo "--- Persistence: umount roi mount lai (du lieu phai con nguyen) ---"
--- Persistence: umount roi mount lai (du lieu phai con nguyen) ---
/ # umount /mnt/jffs2
/ # mount -t jffs2 /dev/mtdblock2 /mnt/jffs2 && echo "REMOUNT OK"
jffs2: Empty flash at 0x00000218 ends at 0x00000400
REMOUNT OK
/ # cat /mnt/jffs2/logs/boot.log
Sat Oct 17 00:03:43 UTC 2026
Written at Sat Oct 17 00:14:20 UTC 2026
/ # cat /mnt/jffs2/runtime.txt
New runtime file
/ # ls -la /mnt/jffs2/
total 2
drwxr-xr-x    4 root     0                0 Jan  1  1970 .
drwxr-xr-x    3 root     0               60 Oct 17 00:13 ..
-rw-r--r--    1 root     0               18 Oct 17 00:03 hello.txt
-rw-r--r--    1 root     0               20 Oct 17 00:03 info.txt
drwxr-xr-x    2 root     0                0 Oct 17 00:03 logs
-rw-r--r--    1 root     0               17 Oct 17 00:14 runtime.txt
/ # umount /mnt/jffs2
/ # echo "--- [C.5] Dump MTD raw data (OOB + 512 byte dau), tim magic JFFS2 0x19
85 ---"
--- [C.5] Dump MTD raw data (OOB + 512 byte dau), tim magic JFFS2 0x1985 ---
```

Hình 5: Mount JFFS2 thành công, đọc file gốc, ghi thêm `runtime.txt` và dòng log mới; remount vẫn giữ nguyên toàn bộ dữ liệu.

`nanddump -o -l 512` dump 512 byte dữ liệu + OOB; `hexdump -C` cho thấy ngay offset 0 là **`85 19`** – magic bitmask của node JFFS2, xác nhận image được ghi đúng vào flash.

```
PHAN C – nanddump raw + magic JFFS2 0x1985
/ # hexdump -C /tmp/dump.bin | head -20
00000000  85 19 01 e0 31 00 00 00  1d d9 82 42 01 00 00 00  |....1......B....|
00000010  00 00 00 00 02 00 00 00  5f bb d2 6a 09 08 00 00  |........_..j....|
00000020  13 93 47 d8 15 da 69 f4  68 65 6c 6c 6f 2e 74 78  |..G...i.hello.tx|
00000030  74 ff ff ff 85 19 02 e0  56 00 00 00 09 64 e7 62  |t.......V....d.b|
00000040  02 00 00 00 01 00 00 00  a4 81 00 00 00 00 00 00  |................|
00000050  12 00 00 00 5f bb d2 6a  5f bb d2 6a 5f bb d2 6a  |...._..j_..j_..j|
00000060  00 00 00 00 12 00 00 00  12 00 00 00 00 00 00 00  |................|
00000070  ee dd 4b b3 8f 63 88 d0  48 65 6c 6c 6f 20 66 72  |..K..c..Hello fr|
00000080  6f 6d 20 4a 46 46 53 32  21 0a ff ff 85 19 01 e0  |om JFFS2!.......|
00000090  30 00 00 00 78 be 3e fa  01 00 00 00 01 00 00 00  |0...x.>.........|
000000a0  03 00 00 00 5f bb d2 6a  08 08 00 00 88 29 36 55  |...._..j.....)6U|
000000b0  a9 de 79 72 69 6e 66 6f  2e 74 78 74 85 19 02 e0  |..yrinfo.txt....|
000000c0  58 00 00 00 3a 13 38 82  03 00 00 00 01 00 00 00  |X...:.8.........|
000000d0  a4 81 00 00 00 00 00 00  14 00 00 00 5f bb d2 6a  |............_..j|
000000e0  5f bb d2 6a 5f bb d2 6a  00 00 00 00 14 00 00 00  |_..j_..j........|
000000f0  14 00 00 00 00 00 00 00  8a 74 43 dd 0b 34 24 8d  |.........tC..4$.|
00000100  45 6d 62 65 64 64 65 64  20 4c 69 6e 75 78 20 4c  |Embedded Linux L|
00000110  61 62 32 0a 85 19 01 e0  2c 00 00 00 5f 56 f1 e0  |ab2.....,..._V..|
00000120  01 00 00 00 02 00 00 00  04 00 00 00 5f bb d2 6a  |............_..j|
00000130  04 04 00 00 29 32 aa 4b  40 19 cb d1 6c 6f 67 73  |....)2.K@...logs|
/ # echo "--- Cleanup: go nandsim ---"
```

Hình 6: Raw dump MTD; magic JFFS2 85 19 ở 0x00 và tên file `hello.txt`, `info.txt`, `logs` xuất hiện trong các dirent node.

Phần D – Tích hợp BusyBox Init & auto-load module

`/etc/init.d/rcS` (chạy ở action `sysinit`) mount `/proc`, `/sys`, `/tmp`, `/dev` rồi `insmod` module và tạo node `/dev/lab2` major 240. Nhờ vậy khi shell mở ra, module đã sẵn sàng.

```
PHAN D – Boot QEMU: rcS auto-load lab2_driver
Run /sbin/init as init process
[ LAB-02 ] Starting system initialization...
input: ImExPS/2 Generic Explorer Mouse as /devices/platform/bus@40000000/bus@40000000:motherboard-bu
s@40000000/bus@40000000:motherboard-bus@40000000:iofpga@7,00000000/10007000.kmi/serio1/input/input2
[ LAB-02 ] Loading lab2_driver module...
lab2_driver: loading out-of-tree module taints kernel.
lab2_driver: initializing module
lab2_driver: /proc/lab2_info created successfully
lab2_driver: loaded successfully, major=240
[ LAB-02 ] lab2_driver loaded OK
[ LAB-02 ] /dev/lab2 created (major=240, minor=0)
[ LAB-02 ] System initialization complete!
  -> Run test_driver.sh  to test character device
  -> Run test_procfs.sh  to test proc/sysfs interfaces
  -> cat /proc/lab2_info to view driver stats

Please press Enter to activate this console.
```

Hình 7: Boot log QEMU – rcS báo `lab2_driver loaded OK`, tạo `/dev/lab2` (major=240, minor=0) trước khi mở shell.

Trong shell BusyBox, các kiểm tra Mục D.5 đều đạt: `lsmod` thấy module, `/dev/lab2` tồn tại, hai script test PASSED, `cat /proc/lab2_info` và các attribute sysfs đọc đúng, `dmesg | grep lab2` in đủ vòng đời module.

```
PHAN B – procfs + sysfs
  -> Run test_procfs.sh  to test proc/sysfs interfaces
  -> cat /proc/lab2_info to view driver stats

Please press Enter to activate this console.
/ # random: fast init done
echo "===== [Kiem tra 3] procfs / sysfs ====="
===== [Kiem tra 3] procfs / sysfs =====
/ # test_procfs.sh
=== Test procfs & sysfs ===
--- /proc/lab2_info (before write) ---
LAB-02 Driver Statistics
======================
Driver name   : lab2
Major number  : 240
Buffer size   : 1024 bytes
Data length   : 0 bytes
Open count    : 0
Last data     : (empty)
lab2_driver: device opened (count=1)
lab2_driver: received 30 bytes: [Embedded Linux Lab2 Test Data
]
lab2_driver: device closed
--- /proc/lab2_info (after write) ---
LAB-02 Driver Statistics
======================
Driver name   : lab2
Major number  : 240
Buffer size   : 1024 bytes
Data length   : 30 bytes
Open count    : 1
Last data     : [Embedded Linux Lab2 Test Data
]
--- sysfs attributes ---
buffer_len : 30
open_count : 1
last_data  : Embedded Linux Lab2 Test Data
=== procfs/sysfs Test PASSED ===
```

Hình 8: (tái hiện) procfs/sysfs đọc trực tiếp trong QEMU shell.

> **Vấn đề gặp phải và cách khắc phục (đáng chú ý):** Ban đầu initramfs của LAB-02 được đóng gói mà **không dùng `fakeroot`**, nên `/dev/console` bị lưu thành *regular file* thay vì character device `c 5 1`. Hệ quả: kernel mở `/dev/console` (fd 0/1/2 của PID 1) trỏ vào file thường nên **toàn bộ output của rcS và shell biến mất khỏi cổng serial** dù module vẫn nạp. Cách sửa: đóng gói lại bằng `fakeroot` + `mknod` để archive chứa đúng device node (giống LAB-01). Sau khi sửa, console serial hoạt động trở lại.

Mục 4: So sánh Filesystem cho hệ thống nhúng

Tiêu chí	ext4	JFFS2	cramfs	SquashFS	UBIFS
Ghi được (writable)	Có	Có	Không (RO)	Không (RO)	Có
Wear leveling	Không tích hợp (nhờ FTL/eMMC)	Có (log-structured)	Không	Không	Có
Nén	Không (cần ext4 + fscrypt/khác)	Có (zlib/rtime)	Có (zlib, block nén)	Có (gzip/lzo/xz/zstd)	Có

Journaling / phục hồi	Journaling	Log-structured (tự phục hồi)	Không (RO)	Không (RO)	Journal (log)
Phù hợp thiết bị	eMMC/SD/HDD/SSD	Raw NAND/NOR nhỏ–vừa	Firmware ROM, rootfs RO	Rootfs RO nén, OTA image	NAND lớn, cần hiệu năng
Thời gian mount	Nhanh (nhưng cần block layer)	Chậm dần khi FS lớn (quét toàn bộ)	Rất nhanh	Rất nhanh (random access nén)	Nhanh (nhờ anchor/UBI)
Hạn chế chính	Không dùng trực tiếp trên raw flash	Mount chậm + RAM cho bảng node khi FS lớn	Chỉ đọc, kích thước block cố định	Chỉ đọc (cần overlay để ghi)	Cần UBI, không hợp NOR/FS < 4 MB

Khuyến nghị cho thiết bị IoT 32 MB NAND cần update firmware OTA:

- **Rootfs (hệ điều hành):** dùng **SquashFS** (read-only, nén tốt, mount nhanh, phù hợp image OTA bất biến) đặt trên phân vùng NAND/UBI, hoặc **UBIFS** nếu muốn rootfs ghi được.
- **Dữ liệu cấu hình/ghi runtime:** dùng **JFFS2** (hoặc UBIFS) trên phân vùng NAND còn lại vì có wear-leveling tích hợp, tránh hỏng flash.
- **OTA:** giữ 2 phân vùng A/B (dual-bank), mỗi bản một image SquashFS, ghi bản mới rồi chuyển boot – vừa an toàn (rollback) vừa tận dụng tính read-only của SquashFS. Không dùng `ext4` trực tiếp trên raw NAND.

Mục 5: Trả lời câu hỏi phân tích

> **Câu 1. Tại sao không thể dùng `memcpy()` thay `copy_to_user()/copy_from_user()`?**

Trong kiến trúc ARMv7 có MMU, mỗi tiến trình user có **không gian địa chỉ ảo riêng** (bảng trang riêng); kernel space nằm ở vùng địa chỉ riêng và được ánh xạ cho mọi tiến trình. Con trỏ `char __user *buf` là **địa chỉ ảo user** – chỉ có ý nghĩa với bảng trang của tiến trình đang gọi. Nếu kernel `memcpy()` trực tiếp con trỏ này:

- Nếu trang đó chưa được nạp vào RAM (page fault), CPU sẽ nhảy vào **page fault handler trong kernel context**; `memcpy()` không có cơ chế xử lý an toàn nên có thể **Kernel Oops / panic**, hoặc ghi sai chỗ.
- Con trỏ có thể là `NULL`, trỏ vào vùng chỉ đọc, hoặc cố tình trỏ vào địa chỉ kernel → **leo thang đặc quyền** (user đọc/ghi được bộ nhớ kernel).
- Trên các CPU có SMAP/PAN, kernel bị cấm truy cập trực tiếp user page → fault ngay.

`copy_to_user()/copy_from_user()` thực hiện theo đúng quy trình: kiểm tra vùng địa chỉ (`access_ok()`), dùng **cơ chế copy đặc biệt có bắt page fault** (qua exception table `__ex_table`), và **trả về số byte chưa copy** (khác 0) để driver trả `-EFAULT` thay vì treo máy. Trong bài lab, nhờ dùng đúng API mà thao tác ghi/đọc `/dev/lab2` luôn nhận đúng 30 byte và không hề gây Oops.

> **Câu 2. Vì sao cần `DEFINE_MUTEX(lab2_mutex)`? Race condition nếu bỏ mutex?**

`device_buffer`, `buffer_len`, `open_count` là tài nguyên dùng chung giữa nhiều tiến trình/CPU. Nếu không có mutex, hai process gọi `write()` đồng thời có thể bị **đan xen (interleaving)**:

```
P1: write("AAAAAAAAAA")                 P2: write("BBBBBBBBBB")
copy_from_user(device_buffer,"AAAAAAAAAA",10)   // đang ghi
                                        copy_from_user(device_buffer,"BBBBBBBBBB",10)
buffer_len = 10   // P1
                                        buffer_len = 10   // P2 (ghi đè)
device_buffer[10]='\0'                  device_buffer[10]='\0'
```

Kết quả có thể là dữ liệu bị **trộn lẫn** (nửa A nửa B), hoặc `buffer_len` không khớp với nội dung thực, khiến process đọc ra **rác / dữ liệu rác giữa các phiên**. Với `open_count++` không khóa, thao tác *read-modify-write* của hai CPU có thể mất một lần tăng (lost update). Mutex đưa các process đến sau vào trạng thái **sleep** (không busy-wait) cho đến khi lock được giải phóng, đảm bảo tính **nguyên tử** và nhất quán. Trong log thực tế, `open_count` tăng liên tục và đúng 1→8 dù nhiều lần mở/đóng – bằng chứng vùng găng được bảo vệ đúng.

> **Câu 3. So sánh procfs (/proc) và sysfs (/sys). Vì sao nên ưu tiên /sys?**

procfs ra đời trước, chủ yếu để xuất trạng thái tiến trình và thông tin kernel dạng text tự do; **sysfs** là hệ thống file ảo có cấu trúc **phân cấp theo mô hình thiết bị (kobject)**. Kernel documentation khuyến nghị **không thêm file tùy tiện vào /proc** vì:

- **ABI không ổn định:** nội dung /proc là text tự do, dễ thay đổi và khó kiểm soát; sysfs có quy tắc rõ ràng (mỗi file một giá trị).

- **Thiếu cấu trúc/khả năng gán:** sysfs ánh xạ 1-1 với device/class/driver, hỗ trợ quyền, symlink, uevent và tự động xuất hiện/biến mất theo vòng đời thiết bị.
- **Nguyên tắc "one value per file":** giúp script/userspace dễ parse, ổn định, tránh việc driver tự định dạng chuỗi.

Với driver bài lab: **thống kê tổng hợp** (`/proc/lab2_info`: tên driver, major, buffer size, data length, open count, last data) phù hợp đặt ở /proc vì đây là dữ liệu "nhiều dòng, mang tính báo cáo". Ngược lại, các **thuộc tính đơn trị có thể đọc/ghi theo từng cái một** (`buffer_len`, `open_count`, `last_data`) đặt ở sysfs; chúng tuân thủ one-value-per-file và tự gắn vào `kobject` của thiết bị nên dễ quản lý bằng script/udev.

> **Câu 4. Vì sao JFFS2 phải erase trước khi ghi, còn ext4 không?**

Đặc tính vật lý của flash NAND/NOR khác đĩa từ/SSD có FTL: (1) ghi chỉ đổi bit **1 → 0**, không đổi được 0 → 1; (2) đơn vị xóa là **eraseblock** (16–256 KiB), không xóa từng byte; (3) số chu kỳ xóa giới hạn. Vì vậy không thể "ghi đè tại chỗ" như ext4 (ghi trực tiếp lên sector bất kỳ qua block layer). Muốn ghi giá trị mới cho một vùng, trước hết phải **erase cả eraseblock về 0xFF**. Trong bài lab, sau `flash_eraseall /dev/mtd2` khi mount nhầm phân vùng chưa format, kernel báo *"Magic bitmask 0x1985 not found ... 0xffff"* và *"Cowardly refusing to erase blocks on filesystem with no valid JFFS2 nodes"* – minh họa rõ rằng vùng flash sạch là 0xFF và JFFS2 cần node hợp lệ mới mount được.

JFFS2 giải quyết bằng thiết kế **log-structured**: mỗi thay đổi được ghi thành một **node mới** (kèm CRC và version) vào chỗ flash trống thay vì sửa node cũ; node cũ bị đánh dấu "obsolete". Khi hết chỗ, **garbage collector** chọn eraseblock có nhiều node cũ, copy các node còn sống sang block khác rồi mới erase block đó. Nhờ ghi trải đều và chỉ ghi tăng thêm, JFFS2 có **wear-leveling** và **bad-block management** tích hợp, phù hợp raw flash nhỏ.

> **Câu 5. Nếu `insmod` trong `rcS` thất bại, hệ thống có boot tiếp không?**

Có. BusyBox init chạy các dòng trong `/etc/inittab` tuần tự; `rcS` chỉ là một process được gọi ở action `sysinit`. Nếu một lệnh trong rcS thất bại, script **vẫn tiếp tục các lệnh sau** (trừ khi script đặt `set -e` hoặc tự `exit`), và init vẫn chuyển sang mở shell ở dòng `::askfirst:-/bin/sh`. Nói cách khác, **lỗi load module mặc định là non-critical**: hệ thống vẫn boot, chỉ thiếu chức năng của module. Chỉ các lỗi **nghiêm trọng** như `mount` thất bại với rootfs, hoặc `exec /sbin/init` thất bại, mới làm kernel panic/dừng boot.

Để phân biệt mức độ, script cần **kiểm tra mã trả về của từng bước quan trọng** và quyết định rõ ràng. Trong bài lab, `rcS` đã kiểm tra `$?` sau `insmod` và in cảnh báo *"ERROR: insmod lab2_driver failed (non-critical, continuing boot)!"* nhưng vẫn tiếp tục – đúng với triết lý "degrade gracefully". Có thể cải thiện như sau:

```
#!/bin/sh
# Ham bao loi nghiem trong: ghi log va dung boot
die() { echo "[ LAB-02 ] FATAL: $1"; exec /bin/sh; }   # hoac: reboot -f

mount -t proc  none /proc || die "khong mount duoc /proc"
mount -t sysfs none /sys  || die "khong mount duoc /sys"
mount -t devtmpfs none /dev 2>/dev/null || mdev -s

KO=/lib/modules/5.15.0/lab2_driver.ko
if [ -f "$KO" ]; then
    if insmod "$KO"; then
        mknod /dev/lab2 c 240 0 2>/dev/null; chmod 666 /dev/lab2
    else
        echo "[ LAB-02 ] WARN: nap $KO that bai (loi ${?}) - tiep tuc boot o che do suy
giam"
        # co the thu: depmod -a; modprobe lab2_driver
    fi
else
    echo "[ LAB-02 ] WARN: thieu $KO - driver khong kha dung"
fi
# Dong cuoi: chi bao thanh cong neu module da nam trong lsmod
lsmod | grep -q '^lab2_driver' \
  && echo "[ LAB-02 ] OK: driver san sang" \
  || echo "[ LAB-02 ] CANH BAO: driver KHONG hoat dong"
```

Nguyên tắc: lỗi **có thể phục hồi** (module/thiết bị phụ) thì log cảnh báo và tiếp tục; lỗi **không thể phục hồi** (rootfs, console, init) thì dừng/khởi động lại để tránh hệ thống chạy nửa vời.

Mục 6: Nhận xét & Khó khăn

Kết quả đạt được: module nạp/gỡ đúng, open/read/write/release hoạt động, procfs + sysfs xuất đúng thống kê, nandsim + JFFS2 mount/ghi/persistence thành công, module được auto-load khi boot.

1. **Khó khăn 1 – console biến mất sau khi đóng gói initramfs:** module vẫn nạp nhưng không thấy output rcS/shell. Nguyên nhân là archive không dùng `fakeroot` nên `/dev/console` là regular file. *Cách giải quyết:* dùng `fakeroot` + `mknod` để đóng gói `/dev/console` (c 5 1), `/dev/null`, `/dev/zero`, `/dev/random` thành character device – giống cách LAB-01 đã làm.
2. **Khó khăn 2 – không `modprobe nandsim` được trên host:** kernel host đang chạy (7.2.9-1) thiếu thư mục module và không có quyền root. *Cách giải quyết:* bật `CONFIG_MTD_RAW_NAND=m` + `CONFIG_MTD_NAND_NANDSIM=m`, build `nand.ko`/`nandsim.ko` kèm kernel target và chạy MTD/JFFS2 ngay trong QEMU ARM; đồng thời xác định NAND simulator là `mtd2` nhờ `/proc/mtd` thay vì giả định `mtd0`.
3. **Khó khăn 3 – build kernel lỗi `dmb ish`:** do thiếu cờ `-mfloat-abi=soft` khiến `cc-option -march=armv7-a` thất bại và rớt xuống `armv5t`. *Cách giải quyết:* build với `CC="arm-linux-gnueabihf-gcc -mfloat-abi=soft"` (đúng như kernel ARM yêu cầu).